

Research note: practising Low-Level Design

1. The learner's problem

Low-Level Design (LLD) rounds ask a candidate to turn a small, well-scoped problem (parking lot, elevator, vending machine) into classes, interfaces, relationships and behaviour, then defend their trade-offs. Learners describe three recurring difficulties:

1. **No way to tell whether a design is good.** Most LLD problems have many valid answers. A learner who compares their design with a published "solution" either copies it or concludes they were wrong because theirs looks different.
2. **Watching feels like learning.** Reading worked solutions produces false confidence ("they look obvious"). Practitioners consistently advise attempting 5–10 problems *without* looking at solutions first. That only works if something else gives feedback.
3. **The interview grades reasoning, not diagrams.** Interviewers probe requirement coverage, where responsibilities sit, what changes when a requirement changes (extensibility), and follow-ups such as concurrency. A static diagram doesn't show any of this unless the learner is asked to write it down.

2. What exists today (surveyed)

Approach	Examples	Strength	Gap for repeated practice
Curated solution libraries	GitHub "awesome-low-level-design" style repos, blog series, YouTube playlists	Free, broad coverage	One reference answer per problem; no feedback on <i>your</i> design
Structured courses	Grokking-style OOD courses, AlgoMaster LLD, Hello Interview "LLD in a Hurry"	Teach patterns and a method	Mostly read-then-compare; feedback is either absent or a premium add-on
Code-first judges / playgrounds	LLD "machine coding" sites, playgrounds with AI code review	Real code, sometimes runnable	Test <i>behaviour</i> or code style; say little about responsibilities, coupling or trade-offs, and nothing when the learner has only a design
Guided practice with AI feedback	Newer interview-prep platforms that add personalised AI feedback	Interactive	Feedback is a black box: hard to know why a point was raised or whether it's reliable, and there is little tracking of improvement across attempts
Generic chat assistants	Pasting a design into an LLM	Instant, flexible	Inconsistent between runs, can hallucinate classes the learner never wrote, no rubric, no history
Human mocks	Peer/paid mock interviews	Best signal	Expensive, not repeatable daily

(Survey based on the public pages of these products and practitioner write-ups. Internal details of commercial products are not claimed.)

3. Key gaps

- **No structured submission format.** Free text and images can't be checked reliably, which is why feedback is either absent or pure LLM opinion.
- **Feedback isn't explainable.** A learner can't tell a hard fact ("FR-5 has no owner") from a judgement call ("this abstraction is premature").
- **Similarity to a reference answer is the wrong metric.** It penalises valid alternatives, which is exactly what a strong candidate needs to learn to defend.
- **No improvement loop.** Tools grade one attempt at a time and don't show what a revision actually fixed.
- **Extensibility is asked about, never tested.** "What if we add EV charging?" is the most common follow-up question. Tools accept any prose answer, but nothing checks whether the design would actually absorb the change.
- **Diagrams are static.** A class diagram can't show whether the classes can collaborate to fulfil a requirement, which is what the "walk me through parking a car" question tests.

4. Product direction

Blueprint is a focused practice loop: *choose* → *design* → *submit* → *feedback* → *review* → *try again*.

1. **A structured design is the submission.** The learner submits classes with responsibilities, relationships, a requirement → class traceability map, patterns with a justification, trade-offs, and an answer to one "what if"

extension scenario. Structure is what makes feedback checkable. It also rehearses what interviewers probe for. Mermaid import/export keeps it fast.

2. **Hybrid, explainable evaluation.** Deterministic rules check facts (coverage, core concepts, god classes, cycles, a missing abstraction at a known point of change, unjustified patterns). An LLM, grounded on those facts, judges quality and suggests alternatives. Every finding is labelled *Rule* or *AI*, points at the learner's own classes, and links back into the editor.
3. **A rubric of properties, not a reference answer.** Checks are phrased as properties ("is pricing behind an abstraction?"), never as specific class names, so different valid designs score well.
4. **Iteration is the unit of progress.** Versions are kept. The report shows what was fixed since the last version, and a progress view shows which criteria are improving.
5. **Test extensibility by changing the requirements.** After feedback, the learner takes a *curveball*: a change request from the problem's deck, or one aimed at the point of change their own design is least ready for. The next report measures the blast radius (classes added vs. existing classes changed). That turns the open/closed principle from a slogan into a measurement.
6. **Make collaboration checkable.** Walkthroughs (ordered calls for a requirement) are validated against the drawn relationships and rendered as sequence diagrams.

Out of scope for the MVP: accounts, an authoring UI, code execution, and a live (human or voice) interviewer. A timed interview mode covers pacing practice.